

Authentication Process — Full Step-by-Step Guide

This guide walks through the **entire authentication process** of this project, from installing JWT to a user successfully logging in and using the app. It covers both the PHP backend (react-project-api) and the React frontend (react-project-php), in the order you would actually build them.

No complicated words — every step is explained in plain language.

What You Will Build

By the end of these steps, your app will be able to: 1. Let a user log in with email and password. 2. Give the user a secure token (JWT) after login. 3. Use that token to allow or block access to pages and data. 4. Log the user out safely.

Part A — Backend Setup (PHP)

Step 1: Install the JWT library

Go to the react-project-api folder in your terminal and run:

```
composer require firebase/php-jwt
```

What this does: - Downloads a small, trusted library that knows how to create and check JWT tokens. - Creates a vendor folder with the library files inside. - Creates or updates composer.json:

```
{
  "require": {
    "firebase/php-jwt": "^7.1"
  }
}
```

Step 2: Connect the library to your project

At the top of any PHP file where you need JWT, load it like this:

```
require_once "../vendor/autoload.php";
use Firebase\JWT\JWT;
use Firebase\JWT\Key;
```

This makes the JWT tools available to use in that file.

Step 3: Create a secret key

The secret key is a private password that only your server knows. It “locks” tokens when they are created and “unlocks” them when they are checked.

Create a file config/secret.php:

```
<?php
```

```

        return [
            'secret_key_jwt' => 'idb70-123456789-abcd.tgrtryuerregtr',
        ];
    }
}
?>

```

Important: Keep this key private. Never send it to the browser or share it publicly. Anyone with this key could create fake tokens.

Step 4: Set up the database connection

Authentication needs a place to check user emails and passwords. This project uses a users table with this structure (config/db.php connects to it):

```

<?php
define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', '');
define('DB_NAME', 'ecom');

$db = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);

if($db->connect_error) {
    die("Connection failed: " . $db->connect_error);
}
?>

```

The users table looks like this:

Column	Type	Purpose
id	int	unique user id
name	varchar	user's name
email	varchar	used to log in
role_id	int	user's role/permission level
password	varchar	the hashed password (never plain text)

Why "hashed"? Passwords are never stored as plain text. They are scrambled using password_hash() when a user is created, so even the developer cannot see the real password. Example from model/user.class.php:

```

$this->password = password_hash($password, PASSWORD_DEFAULT);

```

Step 5: Write the function that creates a token

Create helpers/jwt-helper.php with a function to generate tokens:

```

function generateJWT($payload, $expiry = 3600) {
    global $config;
    $issuedAt = time();
    $expire = $issuedAt + $expiry;
    $tokenPayload = array_merge($payload, [
        'iat' => $issuedAt,
        'exp' => $expire
    ]);
    return JWT::encode($tokenPayload, $config['secret_key_jwt'],
        'HS256');
}

```

Step by step, what happens here: 1. \$payload is the user information you want stored inside the token (like id, name, role). 2. \$issuedAt records the current time. 3. \$expire is the time the token will stop working (default: 1 hour later). 4. iat and exp are added to the payload — these are standard JWT fields meaning “issued at” and “expires at”. 5. JWT::encode() locks everything together using the secret key and returns the final token text.

Step 6: Write the function that checks a token

In the same file, add a function to validate tokens:

```
function validateJWT($jwt) {
    global $config;
    try {
        return JWT::decode(
            $jwt,
            new Key($config['secret_key_jwt'], 'HS256')
        );
    } catch (\Firebase\JWT\SignatureInvalidException $e) {
        return false;
    } catch (\Firebase\JWT\ExpiredException $e) {
        return false;
    } catch (\Exception $e) {
        return false;
    }
}
```

Step by step: 1. Try to unlock (decode) the token using the same secret key. 2. If it unlocks successfully and is not expired, return the information stored inside. 3. If the token was tampered with, return false (SignatureInvalidException). 4. If the token has expired, return false (ExpiredException). 5. If anything else goes wrong, return false.

Step 7: Build the login endpoint

Create api/auth-api.php:

```
function checkLogin($_data){
    echo json_encode(Auth::login($_data['email'],
$_data['password']));
}
```

This function receives the email and password from the request and passes them to the Auth::login() function.

Step 8: Write the login logic

Create model/auth.class.php:

```
class Auth
{
    public static function login($_email, $_password)
    {
        global $db;
        $sql = "select * from users where email = '$_email'";
        $result = $db->query($sql);
        if($result){
            $user = $result->fetch_assoc();
            if($user){
                if(password_verify($_password, $user['password'])){
                    http_response_code(200);
                }
            }
        }
    }
}
```

```

        return [
            "token" => generateJWT($user, 60),
            "user" => $user
        ];
    }else{
        http_response_code(401);
        return "Invalid password.";
    }
}else{
    http_response_code(401);
    return "User not found.";
}
}else{
    return $db->error;
}
}
}

```

Step by step: 1. Look up the user in the database by their email. 2. If no user is found, send back a 401 error ("User not found"). 3. If a user is found, use password_verify() to safely compare the submitted password with the stored hashed password. 4. If the password is wrong, send back a 401 error ("Invalid password"). 5. If the password is correct, call generateJWT(\$user, 60) to create a token, and return the token plus the user's information.

Step 9: Build the main API router with CORS and middleware

Create api/api.php. This file does three jobs: allow requests from the browser (CORS), load all the needed files, and protect routes with the token check.

```

<?php
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE");
header("Access-Control-Allow-Headers: Content-Type, Authorization");

// Handle browser preflight request
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit;
}

require_once "../config/db.php";
foreach(glob("../helpers/*-helper.php") as $helperfile) {
    require_once $helperfile;
}
foreach(glob("../model/*.class.php") as $modalfile) {
    require_once $modalfile;
}
foreach(glob("*-api.php") as $apifile) {
    require_once $apifile;
}

if(!isset($_GET['endpoint']) || $_GET['endpoint'] == "") {
    http_response_code(404);
    echo "<h2>No endpoint found!</h2>";
    exit;
}

$endpoint = $_GET['endpoint'];
$method = $_SERVER['REQUEST_METHOD'];

if($endpoint == "login" && $method == "POST") {

```

```

        $data = json_decode(file_get_contents("php://input"), true);
        checkLogin($data);
    } else {
        // Middleware: check the token before anything else
        $header = getallheaders();
        if(!isset($header["Authorization"])) {
            http_response_code(401);
            echo "Unauthorized. Please login again.";
            exit;
        }
        $jwt = explode(" ", $header["Authorization"]);
        $valid = validateJWT($jwt[1]);
        if (!$valid) {
            http_response_code(401);
            echo "Unauthorized. Please login again.";
            exit;
        }
    }

    // Protected endpoints go here (users, products, categories,
    etc.)

    if ($endpoint == "users" && $method == "GET") {
        getUsers();
    }
    // ...more endpoints
}

```

Step by step: 1. The CORS headers at the top allow the React app (running on a different address) to talk to this PHP server. 2. All helper files (like `jwt-helper.php`) and model files (like `auth.class.php`) are loaded automatically. 3. If the request is for login, it skips the token check (a user without a token needs to be able to log in!) and goes straight to `checkLogin()`. 4. For every other endpoint, it looks for the Authorization header. - No header → blocked with a 401 error. - Header exists → the token is pulled out and checked with `validateJWT()`. - Invalid or expired token → blocked with a 401 error. - Valid token → the request is allowed to continue to the actual endpoint.

Step 10: Set up the URL rewriting

Create `.htaccess` in the root of `react-project-api` so clean URLs work:

```

RewriteEngine On

RewriteRule ^api/(.*)$ api/api.php?endpoint=$1 [L,QSA]

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?page=$1 [L,QSA]

```

This means a request like `http://localhost/react-project-api/api/login` is automatically turned into `api.php?endpoint=login` behind the scenes.

✓ **Backend is now ready.** At this point, if you send a POST request to `/api/login` with an email and password, you should get back a token.

Part B — Frontend Setup (React)

Step 11: Install Axios

Axios is used to send requests to the PHP backend. In your React project folder, run:

```
npm install axios
```

Step 12: Create the shared API connection

Create src/config.ts:

```
import axios from "axios";
import { checkToken } from "../utils/auth";

export const baseApiUrl = "http://localhost/react-project-api/api/";

export const api = axios.create({
  baseURL: baseApiUrl,
  headers: {
    "Content-Type": "application/json",
  },
});
```

This creates one shared connection (api) that every part of the app can use to talk to the backend.

Step 13: Automatically attach the token to every request

Still in config.ts, add this right below the code from Step 12:

```
api.interceptors.request.use((config) => {
  const token = checkToken();
  if(token){
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

What this does: Before every request leaves the app, this code checks if a valid token exists and attaches it to the Authorization header automatically. You never have to add it by hand.

Step 14: Create the token helper functions

Create src/utils/auth.ts:

```
import { redirect } from "react-router";

export function checkToken(){
  const token = localStorage.getItem('bearer_token');
  if(token){
    let payload = JSON.parse(atob(token?.split(".")[1]));
    if(payload.exp * 1000 < Date.now()){
      localStorage.removeItem("bearer_token");
      return false;
    }else{
      return token;
    }
  }else{
    return false;
  }
}
```

```

export const needToLogin = () => {
  const token = checkToken();
  if(!token) throw redirect("/login");
  return null;
};

export const loggedIn = () => {
  const token = checkToken();
  if(token) throw redirect("/");
  return null;
};

```

Step by step: 1. checkToken() reads the token saved in the browser's storage (localStorage). 2. A JWT token has 3 parts separated by dots. The middle part holds information, including the expiry time. atob() decodes it. 3. If the expiry time has already passed, the token is deleted and false is returned. 4. If it's still valid, the token is returned so it can be used. 5. needToLogin() is used to protect pages — if there's no valid token, the user is redirected to /login. 6. loggedIn() does the opposite — if the user is already logged in, they are redirected away from the login page.

Step 15: Build the login page

Create src/views/pages/auth/Login.tsx:

```

import { useState } from "react";
import { useNavigate } from "react-router";
import { api } from "../../../config";

function Login() {
  const [user, setUser] = useState({ email: "", password: "" });
  const [msg, setMsg] = useState("");
  const navigate = useNavigate();

  const handleSubmit = function() {
    api.post("login", user)
      .then((res) => {
        if(res.data.token){
          localStorage.setItem("bearer_token", res.data.token);
          setMsg("");
          navigate("/");
        }
      })
      .catch((err) => {
        if(err.response.status === 401) setMsg(err.response.data);
        else setMsg("⚠ Something went wrong. Login failed!");
      });
  });

  return (
    <form>
      <input
        type="email"
        value={user.email}
        onChange={(e) => setUser({ ...user, email: e.target.value
      )}}
    />
      <input
        type="password"
        value={user.password}
        onChange={(e) => setUser({ ...user, password: e.target.value
      )}}
    />
      <button type="button" onClick={handleSubmit}>Sign In</button>

```

```

        <p>{msg}</p>
      </form>
    );
  }

export default Login;

```

Step by step: 1. The form collects the email and password typed by the user. 2. When the button is clicked, handleSubmit sends a POST request to login with the entered data. 3. If a token comes back, it is saved in localStorage, and the user is sent to the home page. 4. If the login fails, an error message is shown instead (e.g. "Invalid password").

Step 16: Protect pages using route guards

In src/routes.tsx, use needToLogin and loggedIn as **loaders**:

```

import { createBrowserRouter } from "react-router";
import App from "../App";
import Login from "../views/pages/auth/Login";
import { loggedIn, needToLogin } from "../utils/auth";

export const routes = createBrowserRouter([
  {
    path: "/",
    element: <App />,
    loader: needToLogin,
    children: [
      // protected pages go here
    ]
  },
  {
    path: "/login",
    element: <Login />,
    loader: loggedIn
  }
]);

```

What this does: - A "loader" runs before a page is shown. - The home page (/) uses needToLogin, so if there's no valid token, the user is redirected to /login before the page even loads. - The login page (/login) uses loggedIn, so if the user already has a valid token, they get redirected away from the login page (they don't need to log in again).

Step 17: Add a logout button

In src/views/layout/Navbar.tsx:

```

const handleLogout = () => {
  localStorage.removeItem("bearer_token");
  navigate("/login");
};

<button onClick={handleLogout}>Logout</button>

```

Removing the token means checkToken() will now return false, so the next time a protected page is visited, the user gets sent back to /login.

✓ **Frontend is now ready.**

Part C — Test the Whole Process

Follow these steps to confirm everything works together:

1. **Start your PHP server** (e.g. with XAMPP/WAMP or `php -S`).
 2. **Start your React app**: `npm run dev`.
 3. Open the app in the browser — since there's no token yet, you should be redirected to `/login`.
 4. Enter a valid email and password (check the users table for test accounts) and click **Sign In**.
 5. Check the browser's storage (DevTools → Application → Local Storage) — you should see a `bearer_token` key with a long token string.
 6. You should now be redirected to the home page.
 7. Try visiting a protected page or refreshing — you should stay logged in, because the token is still valid.
 8. Click **Logout** — the token disappears from storage, and visiting a protected page sends you back to `/login`.
 9. Wait for the token to expire (or manually delete it) — the app should detect this and ask you to log in again.
-

Full Process Overview (Diagram)

STEP 1-10 Backend (PHP)

Install firebase/php-jwt → Create secret key → Connect database
→ Write `generateJWT()` → Write `validateJWT()` → Build login endpoint
→ Write `Auth::login()` → Build router with CORS + middleware
→ Set up `.htaccess`

STEP 11-17 Frontend (React)

Install axios → Create `config.ts` (API + interceptor)
→ Create `auth.ts` (token check helpers) → Build `Login.tsx`
→ Guard routes with loaders → Add logout button

STEP 18 Full Flow in Action

```
[User submits email/password]
  |
  v
[React sends POST request to /login]
  |
  v
[PHP finds user + verifies password] --wrong--> [401 error shown in React]
  |
  correct
  |
  v
[PHP creates JWT token] --> [Sent back to React]
  |
  v
[React saves token in localStorage]
  |
  v
[Every future request auto-attaches token via interceptor]
  |
  v
[PHP middleware validates token] --invalid/expired--> [401, redirected to login]
  |
  valid
```

|
 v
 [Access granted to protected page/data]
 |
 v
 [User clicks Logout] --> [Token removed] --> [Back to login page]

Quick Reference Table

Step	File	What It Does
1	composer.json	Installs the JWT library
3	config/secret.php	Stores the private secret key
4	config/db.php	Connects to the database
5-6	helpers/jwt-helper.php	Creates and checks tokens
7	api/auth-api.php	Handles the login request
8	model/auth.class.php	Checks email/password and returns a token
9	api/api.php	Routes requests and protects endpoints
10	.htaccess	Enables clean URLs
12-13	src/config.ts	Sets up Axios and auto-attaches the token
14	src/utils/auth.ts	Checks, saves, and expires the token
15	src/views/pages/auth/Login.tsx	The login form
16	src/routes.tsx	Protects pages using loaders
17	src/views/layout/Navbar.tsx	Logs the user out